

STUDY NOTES

Users & Permissions

Complete Guide — Accounts, File Permissions & Access Control

1. Introduction
2. User Accounts
3. Groups
4. File Permissions — The Core Model
5. Changing Permissions & Ownership (chmod, chown, chgrp)
6. Special Permissions — SUID, SGID, Sticky Bit
7. umask
8. Access Control Lists (ACLs)
9. Privilege Escalation — su vs sudo
10. Worked Examples
11. Troubleshooting Checklist
12. Best Practices & Summary

1. Introduction

Every modern operating system is a **multi-user system** at its core — even a personal laptop used by one person is built on the same foundation used by servers with thousands of accounts. To keep data safe and systems stable, the OS must answer two questions for every action:

1. **Who is doing this?** → handled by **Users** (identification and authentication)
2. **Are they allowed to?** → handled by **Permissions** (authorization)

User management decides *who* can log in and what identity they operate under. **Permission management** decides *what* that identity can read, write, or execute.

Together, these form the basis of **access control** — the single most important defense against accidental damage, insider misuse, and external attacks.

Why This Matters

- Prevents one user from reading or deleting another user's files.
- Stops ordinary programs/users from modifying critical system files.
- Limits the damage a compromised account or malware process can do.
- Provides accountability — actions can be traced back to a specific user (auditing).
- Enables safe multi-tenancy on shared servers, cloud instances, and databases.

2. User Accounts

A **user account** is an identity the operating system recognizes, usually backed by a **username** and a **unique numeric ID**.

2.1 Types of Users (Linux/Unix view)

Type	Description	Example
Root / Superuser	Unrestricted account with UID 0 ; bypasses all permission checks	<code>root</code>
Standard (regular) user	Normal human user with limited rights, confined mostly to their own home directory	<code>sam</code> , <code>priya</code>
System / Service account	Non-human account created to run a specific service; usually cannot log in interactively	<code>www-data</code> , <code>mysql</code> , <code>sshd</code>
Guest account	Very limited, temporary access, often disabled by default for security	<code>guest</code>

2.2 Key Identifiers

Term	Meaning
UID (User ID)	A unique number identifying a user internally. UID 0 = root. System accounts typically use 1-999; regular users start from 1000 (Linux) or 500 (older Unix)
GID (Group ID)	The numeric ID of a user's primary group
Username	The human-readable name mapped to a UID
Home directory	The user's private storage area, e.g. <code>/home/sam</code>
Login shell	The program launched at login, e.g. <code>/bin/bash</code> ; set to <code>/usr/sbin/nologin</code> for service accounts to block interactive login

2.3 Where User Information Lives (Linux)

File	Purpose	Who can read
<code>/etc/passwd</code>	Basic account info: username, UID, GID, home dir, shell	World-readable
<code>/etc/shadow</code>	Encrypted password hashes and password-aging rules	Root only
<code>/etc/group</code>	Group names, GIDs, and group membership lists	World-readable
<code>/etc/gshadow</code>	Secure group information (group passwords, admins)	Root only

A `/etc/passwd` line, field by field:

```
sam:x:1001:1001:Sam Verma:/home/sam:/bin/bash
| | | | |
name pw UID GID comment home shell
```

The `x` means the real password hash is stored separately in `/etc/shadow` — this design keeps sensitive hashes out of a world-readable file.

2.4 Common User-Management Commands (Linux)

Command	Purpose
<code>useradd sam</code>	Create a new user
<code>passwd sam</code>	Set or change a password
<code>usermod -aG groupname sam</code>	Add a user to an additional group
<code>userdel -r sam</code>	Delete a user and their home directory
<code>whoami</code>	Show the currently logged-in user
<code>id sam</code>	Show a user's UID, GID, and group memberships
<code>su sam</code>	Switch to another user's session
<code>sudo command</code>	Run a single command with elevated (root) privileges

2.5 Windows Equivalent (Brief Comparison)

Linux/Unix	Windows
UID / GID	SID (Security Identifier)
<code>/etc/passwd</code> , <code>/etc/shadow</code>	SAM database / Active Directory
root	Administrator
Groups	Local Groups / AD Security Groups
<code>chmod</code> / <code>chown</code>	NTFS permissions (via Properties → Security, or <code>icacls</code>)
<code>sudo</code>	"Run as Administrator" / UAC prompt

3. Groups

A **group** is a named collection of users that makes permission management efficient — instead of granting a right to ten individual users, you grant it once to a group they all belong to.

3.1 Primary vs Secondary Groups

- **Primary group** — every user belongs to exactly one; new files they create are owned by this group by default.
- **Secondary (supplementary) groups** — a user can belong to many; used to grant additional shared access (e.g., a `developers` group with write access to a shared project folder).

3.2 Group Commands

Command	Purpose
<code>groupadd developers</code>	Create a new group
<code>groups sam</code>	List groups a user belongs to
<code>gpasswd -a sam developers</code>	Add a user to a group
<code>groupdel developers</code>	Delete a group

3.3 Why Groups Matter

- Simplify administration — one change to the group affects every member.
- Reduce error — no need to remember to update ten separate accounts.
- Model real-world roles: `admins` , `developers` , `hr` , `finance` , `readonly` .

4. File Permissions — The Core Model

Unix-style systems attach three permission categories to every file and directory:

Category	Symbol	Meaning
Owner (User)	<code>u</code>	The user who owns the file
Group	<code>g</code>	Users belonging to the file's group
Others	<code>o</code>	Everyone else

Each category can independently have three types of access:

Permission	Symbol	Effect on a file	Effect on a directory
Read	r	View the file's contents	List the directory's contents
Write	w	Modify or delete the file's contents	Create, delete, or rename files inside it
Execute	x	Run the file as a program/script	Enter (cd into) the directory

4.1 Reading ls -l Output

```
-rwxr-xr-- 1 sam developers 4096 Sep 16 10:00 deploy.sh
```

Segment	Meaning
- (1st char)	File type: - = regular file, d = directory, l = symbolic link
rwx	Owner (sam) permissions: read, write, execute
r-x	Group (developers) permissions: read, execute (no write)
r--	Others' permissions: read only
sam	Owner's username
developers	Owning group

4.2 Octal (Numeric) Notation

Each permission has a fixed numeric weight:

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1
No permission	0

Add the values for each category to get one digit per category (owner, group, others):

Octal	Symbolic	Meaning
7	rwx	Read + Write + Execute
6	rw-	Read + Write
5	r-x	Read + Execute
4	r--	Read only
0	---	No access

Example: `chmod 754 file.sh` → Owner = 7 (rwx), Group = 5 (r-x), Others = 4 (r--) → equivalent to `rwxr-xr--`

4.3 Common Permission Patterns

Octal	Symbolic	Typical use
644	rw-r--r--	Standard file — owner edits, everyone else reads
600	rw-----	Private file — only the owner can read/write (e.g. SSH keys)
755	rwxr-xr-x	Standard program/script or directory — owner controls, others can use/enter
700	rwx-----	Fully private directory
777	rwxrwxrwx	Everyone can do anything — almost always a security risk, avoid

5. Changing Permissions and Ownership

5.1 chmod — Change Mode

Numeric form:

```
chmod 640 report.txt
```

Symbolic form — add (+), remove (-), or set exactly (=):

```

chmod u+x script.sh      # add execute for owner
chmod g-w notes.txt      # remove write for group
chmod o=r data.csv       # set others to read-only
chmod a+r shared.txt     # a = all (u+g+o)
chmod -R 755 /var/www    # apply recursively to a whole directory tree

```

5.2 `chown` — Change Owner

```

chown priya file.txt      # change owner only
chown priya:hr file.txt   # change owner AND group
chown -R priya:hr /project # recursive, whole directory tree

```

5.3 `chgrp` — Change Group Only

```
chgrp developers file.txt
```

Only **root** (or the file owner, in limited cases) can change ownership; regular users can only change permissions on files they already own.

6. Special Permissions

Beyond read/write/execute, three special bits handle edge cases.

6.1 SUID — Set User ID (on executables)

When set on an executable file, it runs with the **file owner's** privileges rather than the privileges of the user who launched it — regardless of who runs it.

- Symbol: an **s** in the owner's execute position (`rwsr-xr-x`)
- Octal: add **4000**
- Classic example: `/usr/bin/passwd` runs as root (via SUID) so any user can update their password entry in the root-only-writable `/etc/shadow`.

```

chmod u+s program
chmod 4755 program

```

Security note: SUID on a writable or poorly-audited program is a major privilege-escalation risk — attackers look specifically for misconfigured SUID binaries.

6.2 SGID — Set Group ID

- **On an executable** — it runs with the privileges of the file's **group**.
- **On a directory** — new files/subfolders created inside automatically inherit the **directory's group** (instead of the creator's primary group). This is extremely useful for **shared team folders**.
- Symbol: an **s** in the group's execute position (`rwxsr-xr-x`)
- Octal: add **2000**

```

chmod g+s /shared/project
chmod 2775 /shared/project

```

6.3 Sticky Bit

Applied to a **directory**: even if a user has write access to the directory, they can only delete or rename files **they own** — not files owned by others.

- Symbol: a **t** in the others' execute position (`rwxxrwxrwt`)
- Octal: add **1000**
- Classic example: `/tmp` — every user can write into it, but no one can delete another user's temp files.

```

chmod +t /tmp
chmod 1777 /tmp

```

6.4 Special Bits Summary

Bit	Octal value	On a file	On a directory
SUID	4000	Run as file owner	(Usually ignored)
SGID	2000	Run as file's group	New files inherit the directory's group
Sticky	1000	(Rarely used today)	Only the owner can delete their own files

7. umask — Default Permission Control

umask is a mask that subtracts permissions from the system default whenever a new file or directory is created, without needing to run `chmod` every time.

- Default base for files: **666** (rw-rw-rw-) — files never get execute by default.
- Default base for directories: **777** (rwxrwxrwx).

Formula: `Resulting permission = Base permission - umask` (bitwise)

Example: `umask 022` - New file: $666 - 022 = 644$ (rw-r--r--) - New directory: $777 - 022 = 755$ (rwxr-xr-x)

```
umask          # view current mask
umask 027      # set a stricter mask for the current session
```

A stricter umask (like `027` or `077`) is common on shared or security-sensitive systems so new files aren't accidentally world-writable.

8. Access Control Lists (ACLs)

The classic owner/group/others model only allows **one** owner and **one** group per file — sometimes real-world needs are more complex (e.g., "Priya should get read-write, but everyone else in Finance should only get read"). **ACLs** extend permissions to multiple specific users or groups on the same file.

Command	Purpose
<code>getfacl file.txt</code>	View the ACL entries on a file
<code>setfacl -m u:priya:rw file.txt</code>	Grant user <code>priya</code> read-write, without changing the base owner/group permissions
<code>setfacl -m g:finance:r file.txt</code>	Grant the <code>finance</code> group read-only access
<code>setfacl -x u:priya file.txt</code>	Remove a specific ACL entry
<code>setfacl -b file.txt</code>	Remove all ACL entries

A file with an ACL shows a `+` at the end of its permission string in `ls -l`:

```
-rw-rwx---+ 1 sam finance 2048 Sep 16 10:00 budget.xlsx
```

9. Privilege Escalation — `su` vs `sudo`

Aspect	<code>su</code>	<code>sudo</code>
Meaning	"Switch User" — opens a full new shell as another user (often root)	"Superuser do" — runs a single command with elevated rights
Password required	The target user's password	The current user's own password
Logging	Minimal by default	Every command is logged to <code>/var/log/auth.log</code> (auditable)
Scope of access	Full session as that user	Configurable per-user, per-command via <code>/etc/sudoers</code>
Best practice	Discouraged for daily admin work	Preferred — least-privilege, traceable

`/etc/sudoers` (edited safely with `visudo`) defines exactly who can run what, as whom:

```
priya ALL=(ALL) ALL          # priya can run any command as any user
deploy ALL=(root) /usr/bin/systemctl restart nginx # deploy can only restart nginx
```

Principle of Least Privilege: Give every account (human or service) only the exact rights it needs to do its job — nothing more. This single idea underlies almost all good permission design.

10. Common Mistakes and Best Practices

Mistake	Why it's a problem	Better approach
<code>chmod 777</code> "just to make it work"	Anyone can read, modify, or delete the file/directory	Diagnose the <i>actual</i> required permission (often 644 or 755)
Sharing the root/Administrator password	No accountability, hard to revoke access for one person	Give individual accounts + <code>sudo</code> rights
Leaving unused/default accounts enabled	Extra attack surface	Disable or delete accounts that are no longer needed
SSH keys or config files world-readable	Secrets can be stolen by any local user	<code>chmod 600</code> on private keys and credential files
Service running as root unnecessarily	A compromised service = full system compromise	Run services under a dedicated, low-privilege service account
No password expiry / weak password policy	Long-lived weak credentials are easy targets	Enforce password aging, complexity, and rotation

Quick Reference — Recommended Permissions

Item	Recommended
Regular document	644
Script/program owner runs	750 or 755
Private keys / secrets	600
Personal home directory	700
Shared team folder (with SGID)	2770
Public temp/shared drop folder	1777

11. Worked Examples

Example 1 — Reading a permission string

```
-rw-r--r-- 1 priya finance 15360 Sep 16 09:00 salary_report.xlsx
```

- File type: `-` → regular file
- Owner `priya`: `rw-` → read and write, no execute
- Group `finance`: `r--` → read only
- Others: `r--` → read only
- In octal:** 644

Question: Can a user named `arjun` (not in the `finance` group) edit this file? *Answer:* No — `arjun` falls under "others," which only has read (`r--`). He can view it but not modify it.

Example 2 — Setting up a shared team directory

Goal: Everyone in the `developers` group should be able to create, edit, and delete files inside `/srv/project`, but users outside the group should have no access at all. New files created inside should automatically belong to the `developers` group.

```
groupadd developers
mkdir /srv/project
chown root:developers /srv/project
chmod 2770 /srv/project
```

- 2** → SGID bit: new files/folders inherit the `developers` group automatically.
- 7** (owner = root) → full access for root.
- 7** (group = developers) → full access for group members.
- 0** (others) → no access at all for anyone else.

Example 3 — Securing a private SSH key

```
ls -l id_rsa
-rw-r--r-- 1 sam sam 1823 Sep 16 09:12 id_rsa
```

This is too open — group and others can read the private key. SSH will actually refuse to use a key with these permissions.

```
chmod 600 id_rsa
```

Now only `sam` (the owner) can read or write it — exactly what a private key needs.

Example 4 — Calculating umask results

If `umask` is set to `027`:

	Base	umask	Result
New file	666 (rw-rw-rw-)	027	640 (rw-r-----)
New directory	777 (rwxrwxrwx)	027	750 (rwxr-x---

This is a common, security-conscious default on multi-user servers — the group can read, but "others" get nothing.

12. Permission Troubleshooting Checklist

When a user reports "Permission denied," walk through this checklist:

1. **Check the exact error** — is it on the file itself, or on a parent directory (need `x` on every directory in the path to reach the file)?
2. **Run `ls -l`** on the file and `ls -ld` on its parent directory to see current owner/group/mode.
3. **Confirm the user's identity and groups** with `id username` — are they actually in the group you expect?
4. **Check for a stricter ACL** with `getfacl` — a base permission can look fine while an ACL entry silently blocks access.
5. **Check special bits** — is a sticky bit preventing deletion, or is SUID/SGID causing unexpected effective privileges?
6. **As a last resort, check SELinux/AppArmor** (on hardened systems) — these mandatory access control layers can deny an action even when standard Unix permissions allow it.

13. Summary

- **Users** establish *identity*; **permissions** establish *authority*.
- Every file has an **owner**, a **group**, and **others**, each with independent **read / write / execute** rights.
- `chmod`, `chown`, `chgrp` are the core tools for adjusting access.
- **SUID, SGID, and the sticky bit** handle special cases beyond the basic model.
- **umask** sets sensible defaults automatically; **ACLs** allow fine-grained exceptions.
- `sudo` (not shared root passwords) is the modern, auditable way to grant elevated access.
- The guiding rule behind every decision should be the **Principle of Least Privilege**: grant the minimum access necessary, for the minimum time necessary.

End of notes — Users & Permissions.